

Fragment Parsing

March 2026

1 Background

1.1 Context-Free Grammars (copied from satta1994)

A *context-free grammar* (CFG) is a four-tuple $G = (N, \Sigma, P, S)$, where N is a finite set of nonterminal symbols, Σ is a finite set of terminal symbols with $N \cap \Sigma = \emptyset$, P is a finite set of productions, and $S \in N$ is the start symbol.

Productions in P have the form $D_r \rightarrow Z_{r,1}Z_{r,2}\dots Z_{r,\pi_r}$, where $D_r \in N$ and $Z_{r,j} \in N \cup \Sigma$, $\pi_r \geq 0$. A production is *null* if $\pi_r = 0$ and a *chain* production if $\pi_r = 1$ and $Z_{r,1} \in N$.

The language derived by G is $L(G) = \{w \mid S \xrightarrow{*} w, w \in \Sigma^*\}$.

1.2 Decomposition

Let ω be an input string that can be decomposed as

$$\omega = \alpha \cdot \beta \cdot \gamma$$

where $\alpha, \gamma \in \Sigma^*$ and $\beta \in \Sigma \cup N$.

There exists a derivation

$$\alpha\beta\gamma \Rightarrow \alpha A \gamma$$

or generally,

$$\alpha\beta\gamma \Rightarrow [\alpha/x]A[\gamma/y]$$

where $x, y \in \Sigma^* \cup N^* \cup \{\varepsilon\}$. The notation $[\alpha/x]$ denotes a correspondence between the input flank α and the production symbol x — specifically, $x \xrightarrow{*} \alpha$ (x derives α). The precise treatment of the recursive case is given in Section 3.

The possible values for the input flanks (α, γ) and production flanks (x, y) are:

I — input flanks (α, γ)	II — production flanks (x, y)
(1) $\alpha = \varepsilon, \gamma = \varepsilon$	(1) $x = \varepsilon, y = \varepsilon$
(2) $\alpha = \varepsilon, \gamma \in \Sigma^+$	(2) $x = \varepsilon$
(3) $\alpha \in \Sigma^+, \gamma = \varepsilon$	(3) $y = \varepsilon$
(4) $\alpha, \gamma \in \Sigma^+$	(4) $x, y \neq \varepsilon$

1.3 Context Reduction Matrix

Mixing and matching cases from I and II gives all possible reduction scenarios for β . Given a production $A \rightarrow x\beta y$ and input fragment $\alpha\beta\gamma$, the grammar defines what flanks are expected. A flank is a *gap* if the production requires it but the input does not supply it.

Two notations are used in the derivations below:

- (x) — x is a production symbol required by the grammar but absent from the input (a gap)
- $[\alpha/x]$ — α is present in the input and corresponds to production symbol x

Cases 1.x — $\alpha = \varepsilon, \gamma = \varepsilon$:

(1.1)	$\beta \rightarrow A$
(1.2)	$\beta \rightarrow \beta(y) \rightarrow A$
(1.3)	$\beta \rightarrow (x)\beta \rightarrow A$
(1.4)	$\beta \rightarrow (x)\beta(y) \rightarrow A$

Cases 2.x — $\alpha = \varepsilon, \gamma$ present:

(2.1)	$\beta\gamma \rightarrow A\gamma$
(2.2)	$\beta\gamma \rightarrow \beta[\gamma/y] \rightarrow A\gamma'$
(2.3)	$\beta\gamma \rightarrow (x)\beta\gamma \rightarrow A\gamma$
(2.4)	$\beta\gamma \rightarrow (x)\beta[\gamma/y] \rightarrow A\gamma'$

Cases 3.x — α present, $\gamma = \varepsilon$:

(3.1)	$\alpha\beta \rightarrow \alpha A$
(3.2)	$\alpha\beta \rightarrow \alpha\beta(y) \rightarrow \alpha A$
(3.3)	$\alpha\beta \rightarrow [\alpha/x]\beta \rightarrow \alpha' A$
(3.4)	$\alpha\beta \rightarrow [\alpha/x]\beta(y) \rightarrow \alpha' A$

Cases 4.x — α present, γ present:

(4.1)	$\alpha\beta\gamma \rightarrow \alpha A\gamma$
(4.2)	$\alpha\beta\gamma \rightarrow \alpha\beta[\gamma/y] \rightarrow \alpha A\gamma'$
(4.3)	$\alpha\beta\gamma \rightarrow [\alpha/x]\beta\gamma \rightarrow \alpha' A\gamma$
(4.4)	$\alpha\beta\gamma \rightarrow [\alpha/x]\beta[\gamma/y] \rightarrow \alpha' A\gamma'$

1.3.1 Note on Null Productions

We assume β is never nullable. For nullable flanks (x or y), an implementation may precompute the nullable set $\mathcal{N}_G = \{A \in N \mid A \xrightarrow{*} \varepsilon\}$ and add skip-rules: for any case where a flank symbol is in $\mathcal{N}_G$, add the corresponding case with that flank dropped. For example, if $x \in \mathcal{N}_G$, case 4.3 ($[\alpha/x]\beta\gamma \rightarrow \alpha' A\gamma$) admits a skip to case 2.1 ($\beta\gamma \rightarrow A\gamma$) directly. Epsilon productions need not fire at runtime.

1.4 Recurrence

Let $R(\omega) \subseteq N$ denote the set of nonterminals that span ω . The recurrence is defined as follows:

Base case:

$$R(\beta) = \{A \in N \mid A \rightarrow x\beta y \in P, \\ x, y \in N \cup \Sigma \cup \{\varepsilon\}\}$$

where x and y are either matched against input flanks or treated as gaps per the CR matrix. This is a fixed-point computation over a finite set — cycles in the grammar do not cause non-termination.

Inductive step: For $\omega = \alpha\beta\gamma$, apply the CR matrix to obtain $\alpha' A\gamma'$ for each $A \in R(\beta)$:

$$R(\omega) = \bigcup_{\{A \in R(\beta)\}} R(\alpha' A\gamma')$$

where $|\alpha' A\gamma'| < |\omega|$, guaranteeing termination.

A *final* nonterminal F is any $A \in R(\omega)$ such that A spans the entire input — i.e. no unreduced symbols remain. F need not be the start symbol S ; any nonterminal covering ω constitutes a valid parse result.

2 Algorithm

2.1 Recognition Table

Let $\omega = w_1 w_2 \dots w_n$ be the input string. The *recognition table* T is an $(n+1) \times (n+1)$ array of sets, where $T[i, j]$ contains *items* representing partial or complete derivations over the span $[i, j]$ (i.e. over $w_{i+1} \dots w_j$).

There are two kinds of items:

- A *complete item* I_A asserts that nonterminal A derives $w_{i+1} \dots w_j$.
- A *partial item* $I_r^{s,t}$ asserts that production r has been partially assembled: symbols $Z_{\{r,s+1\}} \dots Z_{\{r,t\}}$ (the head-adjacent slice) derive $w_{i+1} \dots w_j$, and the remaining symbols $Z_{\{r,1\}} \dots Z_{\{r,s\}}$ and $Z_{\{r,t+1\}} \dots Z_{\{r,\pi_r\}}$ are yet to be combined.

2.2 H-Cover

The *H-cover* $\mathcal{H}(G)$ pre-computes all valid item-to-item relationships from the grammar. It consists of:

- **Projections** (I_A, ξ) : item I_A can be projected from child ξ , where ξ is either a terminal or another item.
- **Left expansions** (I, ξ, I') : item I can be formed by combining left child ξ with right child I' .
- **Right expansions** (I, I', ξ) : item I can be formed by combining left child I' with right child ξ .

The cover is computed once per grammar and reused across all inputs.

2.3 Agenda Algorithm

The algorithm maintains a queue (agenda) of items to process. When item I is added to $T[i, j]$ for the first time, it is pushed onto the agenda.

Processing item I at $T[i, j]$ applies four rules:

1. **Project**: for each $(I', I) \in \text{projections}$, add I' to $T[i, j]$.
2. **Left-expand**: for each $(I', \xi, I) \in \text{left-expansions}$, find all $i' \leq i$ such that $\xi \in T[i', i]$, and add I' to $T[i', j]$.
3. **Right-expand**: for each $(I', I, \xi) \in \text{right-expansions}$, find all $j' \geq j$ such that $\xi \in T[j, j']$, and add I' to $T[i, j']$.
4. **Reverse lookups**: symmetric to 2 and 3 — when I could be the partner already in the table, scan for items waiting on I .

The algorithm terminates because T is finite and each item is enqueued at most once.

2.4 Correctness

Theorem: If the grammar licenses a derivation spanning ω (with possible gaps), the algorithm finds it — i.e. a valid item appears in $T[0, n]$.

The proof proceeds in two parts, one for each fill direction.

2.4.1 Part 1 — Forward (prefix) induction

Claim: If $T[0, k]$ contains a valid item, then $T[0, k+1]$ contains a valid item, provided the grammar licenses one.

Base ($k = 0$): $T[0, 0]$ is seeded with complete items I_A for all $A \in \mathcal{N}_G$ (epsilon nonterminals). Valid by definition.

Base ($k = 1$): $T[0, 1]$ is seeded by projecting w_1 to its licensed items, plus boundary stuffing: for any cover rule $I' \leftarrow \xi I$ where ξ is absent from the input, I' is injected into $T[0, 1]$ with ξ marked virtual. Valid by construction.

Step: Assume $T[0, k]$ contains valid items. Terminal $w_{\{k+1\}}$ seeds $T[k, k+1]$. The agenda combines items from $T[0, k]$ with items from $T[k, k+1]$ via right-expand, producing items in $T[0, k+1]$. The inductive fill then closes over $T[0, k]$: for each item $b \in T[0, k]$, any cover rule licensing b as a right child (with left sibling virtual) adds its parent to $T[0, k]$, which the agenda then propagates into $T[0, k+1]$. Each added item is cover-licensed, so validity is preserved. $\square$

2.4.2 Part 2 — Backward (suffix) induction

Triggered only if $T[0, n]$ is empty after Part 1 — i.e. no full-span item was reached from the left alone.

Claim: If $T[k, n]$ contains a valid item, then $T[k-1, n]$ contains a valid item, provided the grammar licenses one.

Base ($k = n$): $T[n, n]$ is seeded with epsilon items. $T[n-1, n]$ is seeded by projecting w_n plus right-boundary stuffing: for any cover rule $I' \leftarrow I \xi$ where ξ is absent, I' is injected with ξ virtual.

Step: Assume $T[k, n]$ contains valid items. Terminal w_k seeds $T[k-1, k]$. The agenda combines items from $T[k-1, k]$ with items from $T[k, n]$ via left-expand, producing items in $T[k-1, n]$. The inductive fill closes over $T[k, n]$ symmetrically, propagating into $T[k-1, n]$. Validity preserved. $\square$

2.4.3 Convergence

After both passes, a final closure is run on $T[0, n]$ — applying the same fill logic to whatever the two inductions deposited there. Any item reachable by combining prefix and suffix derivations is found at this step.

Together, Parts 1 and 2 guarantee that $T[0, n]$ contains a valid item whenever the grammar licenses a derivation over ω , regardless of which direction the fragment is approached from.

Remark (sequential vs. simultaneous passes). The backward pass is an implementation choice — it fires only if $T[0, n]$ is empty after the forward pass. A simultaneous version (running both directions together, with items from each pass feeding the other) is possible and may find derivations that neither pass finds in isolation. The interaction between the two directions is currently unanalysed.

2.5 Tree Pruning

Recognition populates $T[0, n]$ with items and their derivations. Tree reconstruction follows derivation pointers back through the table, producing a parse forest. For ambiguous grammars or fragment inputs this forest can be exponentially large.

Open problem: the forest must be pruned to be useful. Two criteria are relevant:

1. **Gap minimisation:** trees with fewer virtual (dropped) constituents are better parses of the fragment — a tree with no gaps is a complete parse. Trees should be ranked or filtered by gap count.
2. **Structural plausibility:** among trees with equal gap count, some are more plausible parses than others depending on where the gaps fall (e.g. a missing right argument is more natural than a missing head). A scoring scheme over gap positions is needed.

Neither criterion is currently enforced — the implementation returns all trees up to a fixed cap. A principled pruning strategy is future work.